Pandas aplicado e introducción a SQL

De la pregunta a la consulta

Analítica y Visualización de Datos para la Toma de Decisiones

```
df[df["ciudad"] == "Medellín"]
```

```
SELECT *
FROM ventas
WHERE ciudad = 'Medellín';
```

Estud-IA
HABILIDADES DIGITALES

Alcaldía de Medellín
Distrito de
Ciencia, Tecnología e Innovación

SAPIENCIA
Agencia de Educación Postsecundaria de Medellín

Esumer
Educación Superior

RUTA DE 180 MINUTOS

La sesión combina práctica con Pandas y una introducción a SQL

1

30 min

Inquietudes

Taller anterior

2

50 min

Pandas

Conceptos y ejemplos

3

20 min

Descanso

Pausa intermedia

4

55 min

Notebook

Taller guiado

5

25 min

SQL

Conceptos y ejemplos

Una misma pregunta se resolverá primero con Pandas y luego con SQL.

Alcaldía de Medellín
Distrito de
Ciencia, Tecnología e Innovación

SAPIENCIA
Agencia de Educación
Postsecundaria de Medellín

Esumer
Educación Superior

PRIMEROS 30 MINUTOS

Los errores del taller anterior se revisan mediante un proceso común

```
df["Edaad"].mean()
KeyError: 'Edaad'
```

1 **LEER** última línea

2 **UBICAR** instrucción

3 **COMPARAR** nombre esperado

4 **CORREGIR** y ejecutar

La columna correcta se llama "Edad".

La clase anterior enseñó instrucciones. Hoy las combinamos para analizar tabl

```
df["Edad"].mean()
```

Una columna

```
df[df["ciudad"] == "Medellín"]
```

Un filtro

23 filas

Seleccionar, calcular, filtrar, resumir y ordenar

Estas operaciones forman el núcleo de una consulta sobre datos tabulares.

E-COMMERCE

El caso contiene 96 pedidos realizados por 30 clientes

96
pedidos

30
clientes

12
columnas

3
meses

¿Qué categorías venden más?

¿Qué pedidos superan $300.000?

¿Qué ciudad genera mayor valor?

¿Cuáles son los cinco pedidos principales?

Una tabla organiza observaciones en filas y variables en columnas

pedido_id	ciudad	categoría	cantidad	precio
PED0001	Medellín	Libros	3	95000
PED0002	Medellín	Hogar	1	210000
PED0003	Medellín	Belleza	3	110000
PED0004	Medellín	Tecnología	3	180000
PED0005	Itagüí	Hogar	1	135000

FILAS

pedidos u observaciones

COLUMNAS

variables o características

CELDA

valor de una variable para un pedido

CSV guarda una tabla como texto separado por delimitadores

CSV

```
pedido_id,ciudad,cantidad
PED0001,Medellín,3
PED0002,Bello,1
```

Una tabla por archivo
Sin fórmulas ni formato visual
Fácil de intercambiar entre sistemas

EXCEL

pedido	ciudad	cantidad
PED0001	Medellín	3
PED0002	Bello	1

Puede contener varias hojas
Conserva fórmulas y formatos
Requiere una estructura más compleja

read_csv convierte el archivo en un DataFrame

```
import pandas as pd

df = pd.read_csv("ventas_ecommerce.csv")
```

1 **pd**

alias de Pandas

2 **read_csv()**

función de lectura

3 **df**

DataFrame resultante

El archivo sigue intacto. Pandas crea una representación en memoria.

Alcaldía de Medellín
Distrito de
Ciencia, Tecnología e Innovación
SAPIENCIA
Agencia de Educación Postsecundaria de Medellín
Esumer
Educación Superior

Cada operación responde una pregunta y produce un resultado

1	2	3	4
PREGUNTA	OPERACIÓN	RESULTADO	INTERPRETACIÓN
¿Cuántos pedidos hay?	`len(df)`	96	La tabla contiene 96 pedidos.

Una operación correcta todavía necesita una interpretación correcta.

La inspección inicial confirma tamaño, columnas y tipos

df.head()

primeras filas

df.shape

96 filas, 12 columnas

df.columns

nombres disponibles

df.dtypes

tipo de cada columna

df.info()

estructura y datos faltantes

Inspeccionar antes de calcular

El tipo de dato determina qué operaciones puede ejecutar Pandas

object

texto

```
"Medellín"
```

int64

entero

```
240
```

float64

decimal

```
349016.0
```

datetime64

fecha

```
2026-07-21
```

```
df["fecha_pedido"] = pd.to_datetime(df["fecha_pedido"])
```

Convertir una fecha permite ordenar y calcular periodos correctamente.

Un par de corchetes selecciona una columna. Una lista selecciona varias.

```
df["ciudad"]
```

Serie

Una dimensión

```
df[["ciudad", "categoria_producto"]]
```

DataFrame

Dos dimensiones

La selección reduce la tabla a las variables necesarias para responder una pregunta.

Las funciones de resumen convierten una columna en una medida

sum()	mean()	min()	max()	count()
total	promedio	mínimo	máximo	registros
240 unidades	**$155.760**	**$28.000**	**$650.000**	**96 pedidos**

```
df["cantidad"].sum()
```

Una transformación crea información a partir de columnas existentes

```
df["valor_bruto"] = (
    df["cantidad"] * df["precio_unitario"]
)
```

```
df["valor_neto"] = (
    df["valor_bruto"] - df["valor_descuento"]
)
```

cantidad × **precio** = **valor bruto**

$33.505.500

valor neto total del caso

Una fórmula de negocio se convierte en una nueva variable analítica.

Una condición evalúa cada fila y devuelve verdadero o falso

```
df["valor_neto"] >= 300000
```

pedido	valor neto	>= 300.000
PED0001	285.000	False
PED0004	540.000	True
PED0032	1.428.000	True
PED0007	25.200	False

La condición todavía no elimina filas. Solo las marca.

El filtro utiliza la condición para conservar los registros que cumplen la regla

```
pedidos_altos = df[
    df["valor_neto"] >= 300000
]
```

39 pedidos cumplen la condición

TABLA ORIGINAL	CONDICIÓN	RESULTADO
96 filas	>= 300.000	39 filas

Los operadores lógicos permiten combinar criterios

```
filtro = (
    (df["ciudad"] == "Medellín") &
    (df["valor_neto"] >= 300000)
)

resultado = df[filtro]
```

& **AND**

ambas condiciones

| **OR**

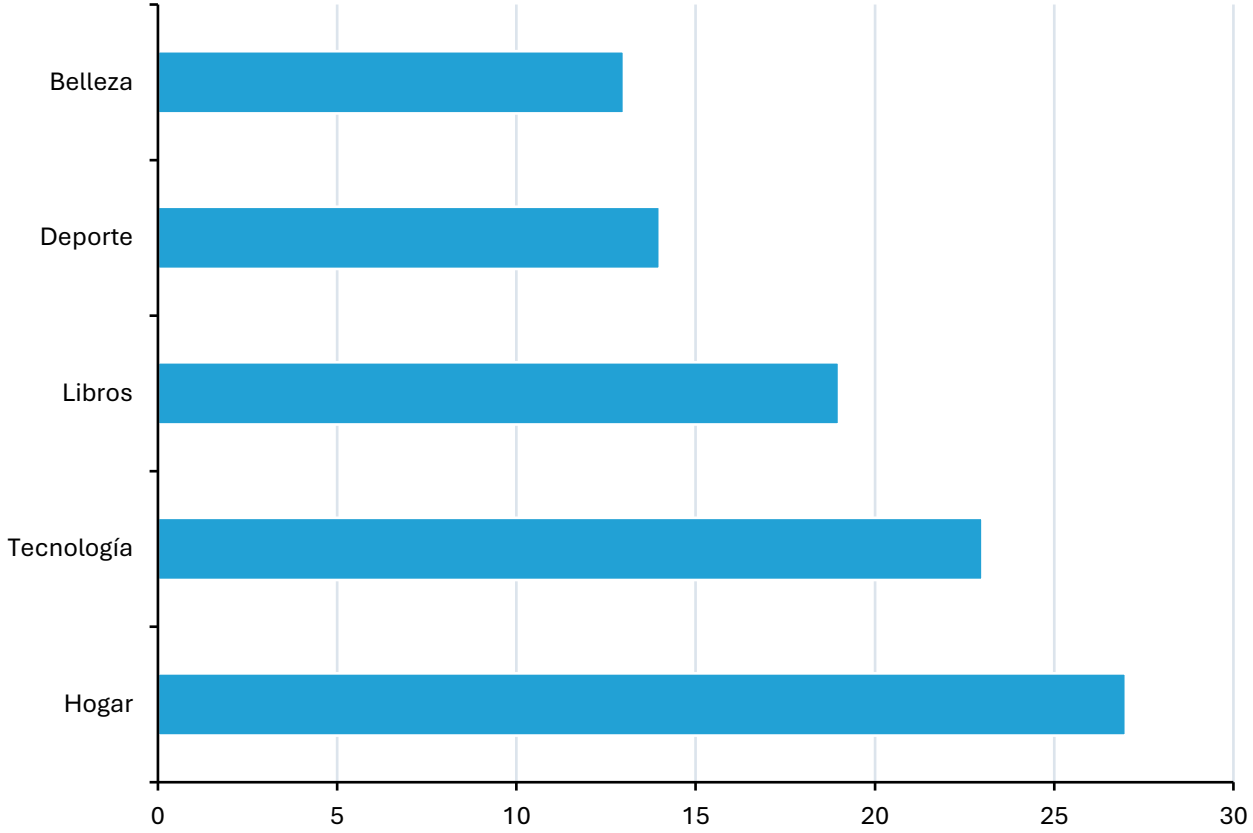
al menos una

~ **NOT**

niega la condición

Resultado: 23 pedidos

value_counts resume cuántas veces aparece cada categoría




```
df["categoria_producto"].value_counts()
```

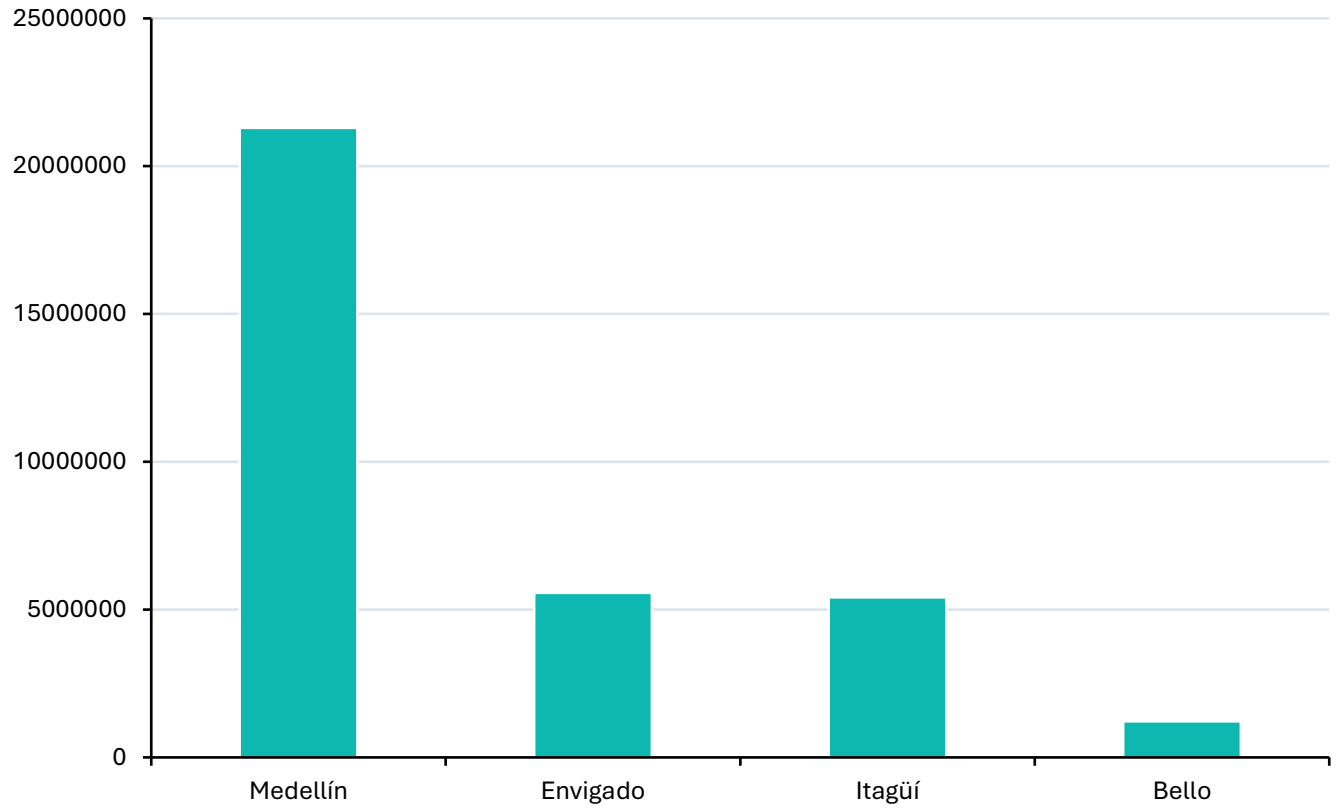
Hogar **27 pedidos**

Es la categoría más frecuente, pero no necesariamente la de mayor valor.

groupby calcula una medida para cada categoría

```
ventas_ciudad = (
    df.groupby("ciudad")["valor_neto"]
      .sum()
      .sort_values(ascending=False)
)
```

Medellín	**$21.298.100**



sort_values y head permiten construir un ranking

```
top_5 = (
    df.sort_values("valor_neto", ascending=False)
      .head(5)
)
```

pedido	ciudad	categoría	valor neto
PED0032	Medellín	Tecnología	$1.428.000
PED0064	Medellín	Hogar	$1.280.000
PED0057	Itagüí	Tecnología	$1.170.000
PED0086	Medellín	Tecnología	$1.134.000
PED0084	Itagüí	Hogar	$1.024.000

PED0032: $1.428.000

Una pregunta de negocio combina varias operaciones

1. **SELECCIONAR** columnas necesarias
2. **CALCULAR** valor neto
3. **FILTRAR** criterios
4. **RESUMIR** por ciudad
5. **ORDENAR** mayor a menor

Pregunta: ¿qué ciudad concentra el mayor valor neto de pedidos entregados?

La secuencia importa porque cada resultado alimenta la operación siguiente.

TALLER GUIADO

El notebook avanza desde la inspección hasta una respuesta de negocio

1 Cargar

2 Inspeccionar

3 Seleccionar

4 Calcular

5 Filtrar

6 Resumir

7 Interpretar

RUTA PRINCIPAL · 55 MIN

Ejemplos resueltos, ejercicios por completar y resultados esperados.

EXTENSIÓN

Reto abierto y comparación con SQL.

Alcaldía de Medellín
Distrito de
Ciencia, Tecnología e Innovación

SAPIENCIA
Agencia de Educación Postsecundaria de Medellín

Esumer
Educación Superior

El reto final exige explicar el resultado, no solo producirlo

A

¿Qué ciudad registra el mayor valor neto?

B

¿Qué categoría tiene más pedidos?

C

¿Cuántos pedidos con descuento superan $300.000?

Entregable: código, resultado, interpretación y una nueva pregunta.

SQL expresa consultas sobre tablas mediante una sintaxis declarativa

```
df[df["ciudad"] == "Medellín"]
```

PANDAS

Indica operaciones sobre un DataFrame

```
SELECT *
FROM ventas
WHERE ciudad = 'Medellín';
```

SQL

Describe el resultado que quiere obtener

La pregunta y el resultado pueden ser los mismos. Cambia el lenguaje.

Una base relacional separa entidades y conecta sus registros

CLIENTES

campo	rol
cliente_id	PK
ciudad	campo
canal_adquisicion	campo
tipo_cliente	campo

1:N

PEDIDOS

campo	rol
pedido_id	PK
cliente_id	FK
fecha_pedido	campo
valor_neto	campo

Un cliente puede realizar varios pedidos.

Cada cláusula de SQL cumple una función dentro de la consulta

```
SELECT pedido_id, valor_neto
FROM ventas
WHERE valor_neto >= 300000
ORDER BY valor_neto DESC
LIMIT 5;
```

SELECT

columnas

FROM

tabla

WHERE

condición

ORDER BY

orden

LIMIT

cantidad

SQL se lee de arriba hacia abajo, aunque la base de datos planifica internamente la ejecución.

Pandas y SQL pueden producir el mismo subconjunto de datos

PANDAS

```
resultado = (
    df[(df["ciudad"] == "Medellín") &
       (df["valor_neto"] >= 300000)]
    .sort_values("valor_neto", ascending=False)
    .head(5)
)
```

SQL

```
SELECT pedido_id, ciudad, valor_neto
FROM ventas
WHERE ciudad = 'Medellín'
  AND valor_neto >= 300000
ORDER BY valor_neto DESC
LIMIT 5;
```

Ambas consultas devuelven los cinco pedidos de Medellín con mayor valor entre los que superan $300.000.

Una consulta comienza con una pregunta clara.

1 — TABLA

2 — OPERACIÓN

3 — RESULTADO

4 — INTERPRETACIÓN

Pandas permite explorar y transformar. SQL permite consultar tablas de una base de datos.

El notebook termina con dos consultas ejecutadas en SQLite.

Alcaldía de Medellín
Distrito de
Ciencia, Tecnología e Innovación

SAPIENCIA
Agencia de Educación
Postsecundaria de Medellín

Esumer
Educación Superior